Module 2_PINN: Physics-Informed Neural Surrogate for Two-Dimensional Electrostatics

Textbook-Style Physics Note for `RadiationEffects_proj2`

Using Module 2 Poisson electrostatics as the first project example of a physics-inspired neural network

---

Purpose of this note. This note introduces physics-informed and physics-inspired neural networks using Module 2 as the concrete example. Module 2 solves the electrostatic Poisson equation that maps radiation-induced defect space charge into electrostatic potential and electric field. The goal here is not to replace the existing finite-element Module 2 implementation. Instead, the goal is to build a parallel field-to-field surrogate that receives an arbitrary spatial total-charge field and predicts the corresponding electrostatic-potential field, while being constrained by the governing electrostatic equation. The note explains, from first principles, what a neural surrogate is, what makes a neural network physics-informed, how the Poisson residual is constructed, how boundary conditions enter the loss function, how automatic differentiation is used, how training data are generated from the existing MATLAB/FEM solver, and how the final trained network can be tested as a fast approximation to Module 2.

Contents

Symbols and acronyms used in this document

Symbol / acronym	Meaning	Typical units
PINN	Physics-informed neural network	–
PDE	Partial differential equation	–

Symbol / acronym	Meaning	Typical units
NN	Neural network	–
MLP	Multilayer perceptron, the standard fully connected feedforward network	–
FEM	Finite-element method	–
AD	Automatic differentiation	–
Ω	Two-dimensional computational domain for Module 2	m^2
$\partial\Omega$	Boundary of the domain	m
Γ_D	Dirichlet boundary segment where potential is prescribed	m
Γ_N	Neumann boundary segment where normal electric flux is prescribed	m
$\boldsymbol{x} = (x, y)$	Spatial coordinate vector	m
$\boldsymbol{\rho}$	Nodal total charge-density field supplied to the field surrogate	$\mathrm{C/m}^3$
$\boldsymbol{\Phi}$	Nodal electrostatic-potential field	V
$\boldsymbol{\Phi}_\theta$	Neural-network prediction of the nodal potential field	V, or dimensionless after scaling
$\mathcal{G}_\theta$	Learned field-to-field electrostatic solution operator	–
$\boldsymbol{\theta}$	Trainable neural-network parameters, weights and biases	dimensionless
$\mathbf{E}$	Electric field, $\mathbf{E} = -\nabla\phi$	V/m
ρ	Total space-charge density	$\mathrm{C/m}^3$
ρ_{def}	Space-charge density due to charged radiation-induced defects	$\mathrm{C/m}^3$
ε, $\varepsilon_{\mathrm{Si}}$	Permittivity, silicon permittivity	F/m
q	Elementary charge magnitude	C
n, p	Electron and hole concentrations	m^{-3}
N_D^+, N_A^-	Ionized donor and acceptor concentrations	m^{-3}
C_i	Concentration of defect species i	m^{-3}
z_i	Effective charge number of defect species i	–

Symbol / acronym	Meaning	Typical units
$\mathcal{R}_{\mathrm{P}}$	Poisson-equation residual used in the PINN loss	depends on scaling
$\mathcal{L}_{\mathrm{data}}$	Supervised data mismatch loss against FEM or reference values	scaled
$\mathcal{L}_{\mathrm{PDE}}$	Interior PDE residual loss	scaled
$\mathcal{L}_{\mathrm{D}}$	Dirichlet boundary loss	scaled
$\mathcal{L}_{\mathrm{N}}$	Neumann boundary loss	scaled
$\mathcal{L}_{\mathrm{BC}}$	Total boundary-condition loss	scaled
$\mathcal{L}_{\mathrm{reg}}$	Regularization or physical-penalty loss	scaled
$\mathcal{L}_{\mathrm{total}}$	Total training objective minimized by the neural network	scaled
N	Number of nodes in one complete field configuration	–
N_F	Number of free FEM degrees of freedom used in the discrete PDE residual	–
N_D	Number of Dirichlet nodes	–
M	Number of complete field configurations in a batch or dataset	–
$\lambda_r, \lambda_D, \lambda_N, \lambda_s$	Loss weights for residual, boundary, and supervised terms	–
L	Characteristic length used for nondimensionalization	m
ϕ_0	Characteristic potential scale	V
ρ_0	Characteristic charge-density scale	$\mathrm{C/m^3}$
$\hat{x}, \hat{y}, \hat{\phi}, \hat{\rho}$	Dimensionless coordinate, potential, and charge density	–

1 Role of this note in the project

The existing Module 2 note develops the electrostatic physics and finite-element implementation for radiation-induced defect space charge in silicon. That physics remains the authoritative governing model. The present note adds a parallel learning-based path:

$$\begin{aligned}&\text{Module 2 FEM field data} + \text{Poisson physics} + \text{boundary conditions}\\ &\longrightarrow \text{trained field-to-field neural surrogate.}\end{aligned} \tag{1.1}$$

The immediate project goal is intentionally limited. Modules 2, 3, 4, and 5 should first be tested separately. Each module should have its own conventional physics implementation and its own neural surrogate implementation. Module 6, the fully coupled multiphysics module, can later combine these pieces. Therefore, the present note focuses only on Module 2:

$$\rho(x, y) \longrightarrow \phi(x, y) \longrightarrow \mathbf{E}(x, y). \tag{1.2}$$

The learning model should be adjacent to the existing Module 2 solver. It should not replace the existing MATLAB/FEM code. The FEM code remains the reference implementation. The neural model is a testbed for three questions:

1. Can a neural network learn the Module 2 electrostatic mapping from charge distribution and boundary conditions to potential?
2. Does adding the Poisson residual and boundary-condition penalties improve physical behavior compared with a purely supervised neural net?
3. Can the trained surrogate predict Module 2 outputs quickly enough to support parameter sweeps or later Module 6 coupling studies?

Important distinction. In this note, *physics-informed* means that the neural-network training objective explicitly penalizes violations of the governing electrostatic equation and boundary conditions. *Physics-inspired* is a broader term. It may include physically motivated inputs, nondimensional variables, monotonicity penalties, positivity constraints, smoothness penalties, or training data generated by a physics solver, even if a full PDE residual is not used. A strict PINN is a physics-informed neural network. A physics-inspired surrogate is a broader class that may or may not be a full PINN.

2 Recap of Module 2 electrostatics

2.1 The physical mapping solved by Module 2

Module 2 solves the electrostatic part of the radiation-effects chain. Radiation creates defects. Some defects carry net charge or alter charge balance. The electrostatic module converts that charge distribution into potential and electric field. The governing relation is Poisson's equation,

$$\nabla \cdot (\varepsilon \nabla \phi) = -\rho. \tag{2.1}$$

For uniform silicon permittivity, this reduces to

$$\varepsilon_{\mathrm{Si}} \nabla^2 \phi = -\rho. \tag{2.2}$$

Once ϕ is known, the electric field is obtained by

$$\mathbf{E} = -\nabla\phi. \tag{2.3}$$

The total space-charge density is

$$\rho = q(p - n + N_D^+ - N_A^-) + \rho_{\text{def}}. \tag{2.4}$$

The defect charge can be represented by

$$\rho_{\text{def}} = q\sum_{i=1}^{M} z_i C_i, \tag{2.5}$$

where C_i is the concentration of defect species i and z_i is its effective charge number.

For a single effective defect species,

$$\rho = q\left(p - n + N_D^+ - N_A^- + z_{\text{def}}C\right). \tag{2.6}$$

2.2 Boundary conditions

For a well-posed electrostatic problem, Poisson's equation must be paired with boundary conditions. On a Dirichlet boundary Γ_D, the potential is prescribed:

$$\phi(\boldsymbol{x}) = \phi_D(\boldsymbol{x}), \qquad \boldsymbol{x} \in \Gamma_D. \tag{2.7}$$

On a Neumann boundary Γ_N, the normal electric displacement flux is prescribed:

$$\mathbf{n} \cdot \varepsilon\nabla\phi(\boldsymbol{x}) = g_N(\boldsymbol{x}), \qquad \boldsymbol{x} \in \Gamma_N. \tag{2.8}$$

Because $\mathbf{E} = -\nabla\phi$, the Neumann condition can also be interpreted as a condition on normal electric field. Care is needed with sign convention. In the form above, g_N is the outward-normal component of $\varepsilon\nabla\phi$, not the outward-normal component of $\varepsilon\mathbf{E}$.

2.3 Why Module 2 is a good first PINN test case

Module 2 is a good first learning-based module for four reasons.

First, the governing PDE is elliptic and comparatively simple. The main equation is Poisson's equation, not a stiff time-dependent reaction network.

Second, the output is a smooth scalar field $\phi(x, y)$ in many practical cases. Neural networks often approximate smooth functions more easily than discontinuous or shock-like solutions.

Third, the electric field can be obtained by differentiating the learned potential. This connects naturally to automatic differentiation.

Fourth, the existing FEM solver can generate reference solutions. This makes it possible to compare a purely supervised neural network, a physics-informed neural network, and the original FEM solver on the same cases.

3 What a neural surrogate means

3.1 A conventional solver versus a field-to-field surrogate

For a fixed geometry, material distribution, and set of boundary conditions, the conventional Module 2 FEM solve takes a complete spatial charge field and returns a complete spatial potential field:

$$\boldsymbol{\rho} \longrightarrow K\boldsymbol{\Phi} = \boldsymbol{b}(\boldsymbol{\rho}) \longrightarrow \boldsymbol{\Phi} \longrightarrow \mathbf{E}_h. \tag{3.1}$$

Here

$$\boldsymbol{\rho} = [\rho_1, \rho_2, \ldots, \rho_N]^T \tag{3.2}$$

contains total charge density at the N FEM nodes, and

$$\boldsymbol{\Phi} = [\Phi_1, \Phi_2, \ldots, \Phi_N]^T \tag{3.3}$$

contains the corresponding nodal electrostatic potential.

The design decision adopted for Module 2 is to learn this *field-to-field* mapping directly:

$$\boxed{\mathcal{G}_\theta : \boldsymbol{\rho} \mapsto \boldsymbol{\Phi}_\theta} \tag{3.4}$$

rather than learning a map from a small vector of Gaussian parameters. The Gaussian model remains useful, but only as one synthetic generator of training fields $\boldsymbol{\rho}$.

This distinction is fundamental. A charge field generated by Module 3 or Module 5 does not need to be compressed into a center, width, or amplitude before Module 2 can use it. If an upstream solver supplies arbitrary nodal fields, Module 2 can consume those fields directly.

3.2 Canonical Module 2 input contract

The total charge field remains

$$\rho(x,y) = q\left[p(x,y) - n(x,y) + N_D^+(x,y) - N_A^-(x,y)\right] + \rho_{\text{def}}(x,y), \tag{3.5}$$

with

$$\rho_{\mathrm{def}}(x,y) = q\sum_i z_i C_i(x,y). \tag{3.6}$$

For the discrete FEM problem, each of these quantities may be represented at the mesh nodes. Module 2 therefore separates two operations:

$$\{p, n, N_D^+, N_A^-, C_i, z_i\} \longrightarrow \boldsymbol{\rho} \longrightarrow \mathcal{G}_\theta(\boldsymbol{\rho}) = \boldsymbol{\Phi}_\theta. \tag{3.7}$$

The first arrow is deterministic charge bookkeeping. The second arrow is the learned electrostatic solution operator.

For a fixed geometry and fixed boundary conditions, $\boldsymbol{\rho}$ is therefore the only spatial physics input required by the core surrogate. If future studies vary permittivity, geometry, or boundary conditions, those quantities can be supplied as additional spatial channels or conditioning variables. For example,

$$\mathcal{G}_\theta : (\boldsymbol{\rho}, \boldsymbol{\varepsilon}, \boldsymbol{m}_{\mathrm{geom}}, \boldsymbol{b}_{\mathrm{BC}}) \mapsto \boldsymbol{\Phi}_\theta. \tag{3.8}$$

The first implementation should keep geometry, material, and electrode conditions fixed so that the field-to-field change is isolated cleanly.

3.3 Why Gaussian parameters are no longer surrogate inputs

A synthetic Gaussian defect packet may still be defined by

$$C_{\mathrm{def}}(x,y) = C_{\mathrm{bg}} + C_{\mathrm{peak}} \exp\left[-\frac{(x-x_c)^2}{2\sigma_x^2} - \frac{(y-y_c)^2}{2\sigma_y^2}\right]. \tag{3.9}$$

The values

$$(C_{\mathrm{peak}}, x_c, y_c, \sigma_x, \sigma_y) \tag{3.10}$$

are now *generator metadata*. They create one example $\boldsymbol{\rho}$ field, but the neural surrogate receives the resulting field, not those five values. Consequently, arbitrary non-Gaussian, multimodal, irregular, imported, or coupled charge maps can enter through the same interface without changing the electrostatic solver.

4 What makes the field-to-field network physics-informed

4.1 Purely supervised field learning

Suppose the FEM generator provides M complete field pairs

$$\left\{\left(\boldsymbol{\rho}^{(m)}, \boldsymbol{\Phi}_{\mathrm{FEM}}^{(m)}\right)\right\}_{m=1}^{M}. \tag{4.1}$$

A purely supervised field surrogate minimizes, for example,

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\frac{1}{N}\left\|\mathcal{G}_\theta\left(\boldsymbol{\rho}^{(m)}\right) - \boldsymbol{\Phi}_{\text{FEM}}^{(m)}\right\|_2^2. \tag{4.2}$$

This objective learns complete configurations. Training, validation, and test splits must therefore be made by *whole field cases*; nodes from one configuration must never be scattered across different dataset splits.

4.2 Physics-informed field learning on the FEM mesh

The strongest connection to the existing Module 2 code is obtained by enforcing the same discrete weak-form physics used by FEM. Before Dirichlet constraints are applied, the electrostatic system is

$$K\boldsymbol{\Phi} = \boldsymbol{b}(\boldsymbol{\rho}), \tag{4.3}$$

where K depends on geometry and permittivity, while $\boldsymbol{b}(\boldsymbol{\rho})$ is assembled from the nodal charge field.

Partition the nodes into free nodes F and prescribed Dirichlet nodes D. Then the exact free-node equation is

$$K_{FF}\boldsymbol{\Phi}_F + K_{FD}\boldsymbol{\Phi}_D = \boldsymbol{b}_F(\boldsymbol{\rho}). \tag{4.4}$$

For a neural prediction $\boldsymbol{\Phi}_\theta$, define the discrete Poisson residual

$$\boxed{\boldsymbol{r}_{\text{PDE}}(\boldsymbol{\rho};\theta) = K_{FF}\boldsymbol{\Phi}_{\theta,F} + K_{FD}\boldsymbol{\Phi}_D - \boldsymbol{b}_F(\boldsymbol{\rho})} \tag{4.5}$$

and a physics loss

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_F}\left\|\boldsymbol{r}_{\text{PDE}}\right\|_2^2, \tag{4.6}$$

with appropriate nondimensionalization or residual scaling as discussed later.

This residual is especially attractive for the present project because it uses the same triangular mesh, material operator, source assembly, and boundary partition already verified in the FEM solver. The neural model is therefore not asked to approximate spatial derivatives independently from the discretization used to create its reference data.

4.3 Strong-form residual remains an optional alternative

A continuous coordinate-conditioned decoder could still enforce

$$\nabla\cdot(\varepsilon\nabla\phi_\theta) + \rho = 0 \tag{4.7}$$

at arbitrary spatial points using automatic differentiation. That remains mathematically valid. It is no longer the preferred baseline for the field-to-field Module 2 architecture, however, because a CNN, U-Net, Fourier neural operator, or mesh/graph operator naturally predicts a discrete field rather than a scalar function of (x, y). For those architectures, the discrete FEM residual in Eq. (4.5) is the cleaner first implementation.

5 Neural-network representation of the potential field

5.1 The network now represents an operator between fields

The target object is no longer a scalar multilayer perceptron of the form $(x, y, \mu) \mapsto \phi$. Instead, the network receives a discretized charge field and returns a discretized potential field:

$$\boldsymbol{\Phi}_\theta = \mathcal{G}_\theta(\boldsymbol{\rho}). \tag{5.1}$$

For a fixed mesh with N nodes,

$$\boldsymbol{\rho}, \boldsymbol{\Phi}_\theta \in \mathbb{R}^N. \tag{5.2}$$

If fields are rasterized to a structured $N_x \times N_y$ array, then the same mapping can be written

$$\rho \in \mathbb{R}^{N_x \times N_y} \longmapsto \phi_\theta \in \mathbb{R}^{N_x \times N_y}. \tag{5.3}$$

5.2 Architecture choices

Several architectures can implement Eq. (3.4):

- A fully connected network can flatten $\boldsymbol{\rho}$ and predict $\boldsymbol{\Phi}$, but its parameter count scales poorly with mesh size and it ignores spatial locality.
- A convolutional network or U-Net is natural when both fields are represented on a regular rectangular grid. The Module 9 geometry then requires a mask or rasterized material/boundary channels.
- A Fourier neural operator is also natural on a structured grid and learns a global field operator efficiently for repeated fixed-domain problems.
- A graph or mesh neural operator can act directly on the existing unstructured triangular FEM mesh, avoiding rasterization and preserving the actual Module 9 connectivity.

The physics note does not require the final architecture to be chosen immediately. What is fixed now is the *contract*: arbitrary spatial charge field in, spatial potential field out.

5.3 Normalization of field inputs and outputs

Field values can span many orders of magnitude. Introduce characteristic scales ρ_0 and ϕ_0 and define

$$\hat{\boldsymbol{\rho}} = \frac{\boldsymbol{\rho}}{\rho_0}, \qquad \hat{\boldsymbol{\Phi}} = \frac{\boldsymbol{\Phi}}{\phi_0}. \tag{5.4}$$

The scales should be computed from the training set only, then frozen and reused for validation, test, and inference. Suitable choices include physically motivated characteristic scales or robust dataset scales such as a high percentile of $|\rho|$ and $|\phi|$.

Coordinates may also be supplied as auxiliary channels, especially for CNN or graph architectures. For example,

$$\hat{x} = \frac{x - x_c^{\mathrm{dom}}}{L_x}, \qquad \hat{y} = \frac{y - y_c^{\mathrm{dom}}}{L_y}. \tag{5.5}$$

These coordinate channels describe where a field value lives; they do not replace the field itself.

6 Training samples, residual degrees of freedom, and complete-case splitting

6.1 The basic training sample is one complete field configuration

For the field-to-field surrogate, one sample is

$$\left(\boldsymbol{\rho}^{(m)}, \boldsymbol{\Phi}_{\mathrm{FEM}}^{(m)}\right), \tag{6.1}$$

not one node $(x_i, y_i, \rho_i, \phi_i)$. This matters because the electrostatic potential at one node depends nonlocally on charge throughout the domain. A node-level random split would therefore leak information from the same physical configuration into both training and test data.

The correct split is

$$\{\text{complete charge fields}\} = \mathcal{D}_{\mathrm{train}} \cup \mathcal{D}_{\mathrm{val}} \cup \mathcal{D}_{\mathrm{test}}, \tag{6.2}$$

with no field configuration appearing in more than one subset.

6.2 Physics residual locations

With the discrete FEM residual, there is no need to create an unrelated cloud of collocation points. The residual is evaluated at the free FEM degrees of freedom using Eq. (4.5). Boundary constraints are evaluated on the tagged Dirichlet or Neumann parts of the same mesh.

If a future coordinate-based decoder uses the strong-form PDE instead, interior collocation points can again be sampled independently of the FEM nodes. That is an optional alternative, not the

baseline field-to-field design.

7 Three levels of neural modeling for Module 2

7.1 Level 1: supervised field-to-field surrogate

Train $\mathcal{G}_\theta$ only on complete FEM field pairs using $\mathcal{L}_{\text{data}}$. This establishes that the chosen architecture and data pipeline can learn the operator $\boldsymbol{\rho} \mapsto \boldsymbol{\Phi}$.

7.2 Level 2: physics-inspired field surrogate

Add physically motivated preprocessing or penalties without yet enforcing the full discrete Poisson residual. Examples include nondimensionalization, exact Dirichlet masking, geometry channels, charge-conservation diagnostics, or electric-field smoothness checks.

7.3 Level 3: physics-informed field surrogate

Add the discrete FEM Poisson residual and boundary constraints directly to the training objective:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_D\mathcal{L}_{\text{D}} + \lambda_N\mathcal{L}_{\text{N}} + \lambda_E\mathcal{L}_E. \tag{7.1}$$

For the static Module 2 electrostatic problem there is no initial-condition loss. An $\mathcal{L}_{\text{IC}}$ term would only be relevant in a time-dependent module.

Recommended near-term path. First validate the new ρ-field FEM input contract and the saved $\rho \to \phi$ dataset. Then implement a supervised field-to-field network. Finally add the differentiable FEM residual and the adaptive loss-weighting procedure described in Chapter 14. This preserves a clean separation between data-pipeline debugging and physics-loss debugging.

8 Detailed construction of the field-to-field PINN loss

8.1 Discrete interior Poisson residual

For each complete field case m, assemble the source vector $\boldsymbol{b}^{(m)} = \boldsymbol{b}(\boldsymbol{\rho}^{(m)})$. The network predicts $\boldsymbol{\Phi}_\theta^{(m)}$. On free nodes,

$$\boldsymbol{r}_{\text{PDE}}^{(m)} = K_{FF}\boldsymbol{\Phi}_{\theta,F}^{(m)} + K_{FD}\boldsymbol{\Phi}_D^{(m)} - \boldsymbol{b}_F^{(m)}. \tag{8.1}$$

A normalized batch loss can be written

$$\mathcal{L}_{\mathrm{PDE}} = \frac{1}{M} \sum_{m=1}^{M} \frac{\|\boldsymbol{r}_{\mathrm{PDE}}^{(m)}\|_2^2}{N_F \, r_0^2}, \tag{8.2}$$

where r_0 is a reference residual scale chosen consistently with the nondimensionalization.

8.2 Dirichlet boundary loss

If Dirichlet values are not imposed exactly in the network output layer, use

$$\mathcal{L}_{\mathrm{D}} = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{N_D} \left\| \boldsymbol{\Phi}_{\theta,D}^{(m)} - \boldsymbol{\Phi}_{D}^{(m)} \right\|_2^2. \tag{8.3}$$

On a fixed transmon geometry with fixed electrode voltages, hard enforcement by overwriting or masking the tagged electrode outputs is particularly attractive because it removes one optimization burden entirely.

8.3 Neumann boundary loss

Homogeneous natural Neumann conditions are already embedded in the standard FEM weak form when no boundary-flux source is assembled. Therefore a separate $\mathcal{L}_{\mathrm{N}}$ is not required for the first discrete weak-form implementation. If nonzero fluxes or a separately evaluated boundary derivative are introduced later, an explicit Neumann mismatch term can be added.

8.4 Supervised FEM data loss

For complete reference fields,

$$\mathcal{L}_{\mathrm{data}} = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{N} \left\| \boldsymbol{\Phi}_{\theta}^{(m)} - \boldsymbol{\Phi}_{\mathrm{FEM}}^{(m)} \right\|_2^2. \tag{8.4}$$

This term anchors the operator to high-fidelity numerical solutions while the PDE loss penalizes physically inconsistent predictions.

8.5 Electric-field consistency loss

The electric field may be recovered from the predicted nodal potential using the same FEM gradient reconstruction used by the reference solver. If G denotes that discrete gradient operator schematically,

$$\mathbf{E}_{\theta} = -G\boldsymbol{\Phi}_{\theta}. \tag{8.5}$$

An optional field loss is

$$\mathcal{L}_E = \frac{1}{M}\sum_{m=1}^{M}\frac{\|\mathbf{E}_\theta^{(m)} - \mathbf{E}_{\mathrm{FEM}}^{(m)}\|_2^2}{N_E E_0^2}. \tag{8.6}$$

This term is optional because $\mathbf{E}$ is already determined by ϕ; it is mainly useful when electric-field accuracy is the downstream quantity of greatest interest.

9 Automatic differentiation in the field-to-field formulation

9.1 What automatic differentiation still does

Automatic differentiation remains essential for training because the optimizer needs

$$\nabla_\theta \mathcal{L}_{\mathrm{total}}. \tag{9.1}$$

The computational graph includes the field-to-field network, the predicted $\boldsymbol{\Phi}_\theta$, the matrix-vector residual $K\boldsymbol{\Phi}_\theta - \boldsymbol{b}$, and the individual loss terms. Automatic differentiation propagates derivatives backward through this graph to every trainable weight and bias.

9.2 What changes relative to the pointwise PINN

In the older pointwise formulation, automatic differentiation was also used to obtain

$$\frac{\partial^2 \phi_\theta}{\partial x^2}, \qquad \frac{\partial^2 \phi_\theta}{\partial y^2}. \tag{9.2}$$

That is not necessary for the preferred field-to-field baseline. The spatial physics is supplied by the already verified FEM operator K and source assembly $\boldsymbol{b}(\boldsymbol{\rho})$. Thus:

$$\boxed{\text{AD for parameter gradients remains; AD for spatial second derivatives becomes optional.}} \tag{9.3}$$

This change is beneficial on the unstructured Module 9 mesh because it avoids pretending that a discrete CNN or mesh-operator output is inherently a continuous scalar function of (x, y).

10 Analytical verification cases for Module 2_PINN

A neural surrogate should not be trusted until it passes simple cases where the answer is known.

10.1 Zero-charge linear-potential case

Let

$$\rho(x, y) = 0 \tag{10.1}$$

in a rectangle $0 \le x \le L_x$, $0 \le y \le L_y$. Apply

$$\phi(0, y) = V_L, \qquad \phi(L_x, y) = V_R, \tag{10.2}$$

and insulating top and bottom boundaries. The exact solution is

$$\phi(x, y) = V_L + \frac{V_R - V_L}{L_x} x. \tag{10.3}$$

The electric field is uniform:

$$\mathbf{E} = -\nabla\phi = -\frac{V_R - V_L}{L_x}\hat{\boldsymbol{x}}. \tag{10.4}$$

This case checks whether the network can satisfy Laplace's equation and simple Dirichlet conditions.

10.2 Uniform charge in one dimension

If the solution depends only on x and the charge density is uniform, then

$$\varepsilon_{\mathrm{Si}} \frac{d^2\phi}{dx^2} = -\rho_0. \tag{10.5}$$

Integrating twice gives

$$\phi(x) = -\frac{\rho_0}{2\varepsilon_{\mathrm{Si}}} x^2 + C_1 x + C_2. \tag{10.6}$$

With $\phi(0) = V_L$ and $\phi(L_x) = V_R$,

$$C_2 = V_L, \tag{10.7}$$

$$C_1 = \frac{V_R - V_L}{L_x} + \frac{\rho_0 L_x}{2\varepsilon_{\mathrm{Si}}}. \tag{10.8}$$

Therefore,

$$\phi(x) = V_L + \left(\frac{V_R - V_L}{L_x} + \frac{\rho_0 L_x}{2\varepsilon_{\mathrm{Si}}} \right) x - \frac{\rho_0}{2\varepsilon_{\mathrm{Si}}} x^2. \tag{10.9}$$

This case checks whether the network learns curvature from space charge.

10.3 Gaussian defect charge

A localized defect-charge source can be defined as

$$\rho(x, y) = A_\rho \exp\left[-\frac{(x - x_c)^2}{2\sigma_x^2} - \frac{(y - y_c)^2}{2\sigma_y^2} \right]. \tag{10.10}$$

This case usually has no simple closed-form solution in a finite rectangle with mixed boundary conditions, but it is a useful FEM-vs-PINN comparison case. The network should reproduce:

- the potential perturbation near the charged region;
- the sign of the field deflection;
- the smoothing effect of Poisson's equation;
- correct boundary potentials.

11 Training modes for the field-to-field implementation

11.1 Mode A: single-field pipeline smoke test

Use one simple charge field and one FEM solution only to verify tensor shapes, normalization, forward prediction, loss evaluation, and backpropagation. This is a software test, not a useful general surrogate, because one field pair cannot teach an operator over many charge configurations.

11.2 Mode B: supervised field-to-field surrogate over many FEM cases

Generate many complete charge fields $\boldsymbol{\rho}^{(m)}$ and paired FEM potentials $\boldsymbol{\Phi}_{\mathrm{FEM}}^{(m)}$. Train with $\mathcal{L}_{\mathrm{data}}$ only. This is the correct baseline against which the physics-informed version should be compared.

11.3 Mode C: physics-informed field-to-field surrogate over many FEM cases

Use the same complete field pairs, but add the discrete FEM residual and boundary treatment. This is the final target architecture for Module 2:

$$\boxed{\boldsymbol{\rho} \xrightarrow{\mathcal{G}_\theta} \boldsymbol{\Phi}_\theta \xrightarrow{K\boldsymbol{\Phi}_\theta - \boldsymbol{b}(\boldsymbol{\rho})} \text{physics residual}} \tag{11.1}$$

with supervised FEM data and the physics losses optimized together.

12 Representing and generating charge fields

12.1 Arbitrary nodal fields are the canonical representation

On the current FEM mesh, the canonical input is

$$\boldsymbol{\rho} \in \mathbb{R}^N, \tag{12.1}$$

one total charge-density value per mesh node. The field may come from any source: a Gaussian synthetic generator, a sum of several localized clusters, a uniform distribution, an imported map, Module 3 defect evolution, Module 5 carrier concentrations, or a coupled Module 6 state.

The electrostatic solver does not need to know how the field was generated. Once the total $\boldsymbol{\rho}$ is supplied, all source mechanisms enter Poisson's equation through the same right-hand side.

12.2 Component fields and total charge

For coupling, it is useful to retain component maps before combining them:

$$\boldsymbol{p}, \quad \boldsymbol{n}, \quad \boldsymbol{N}_D^+, \quad \boldsymbol{N}_A^-, \quad \boldsymbol{C}_i. \tag{12.2}$$

Module 2 then forms

$$\boldsymbol{\rho} = q\left(\boldsymbol{p} - \boldsymbol{n} + \boldsymbol{N}_D^+ - \boldsymbol{N}_A^- + \sum_i \boldsymbol{z}_i \odot \boldsymbol{C}_i\right), \tag{12.3}$$

where $\odot$ denotes pointwise multiplication. Scalars are simply uniform fields in this representation.

12.3 Gaussian distributions are synthetic generators only

The existing Gaussian design remains valuable because it provides controlled variations in amplitude, position, depth, and width. However, its parameters belong in dataset provenance:

$$\boldsymbol{\mu}_{\text{gen}} = [C_{\text{peak}}, x_c, y_c, \sigma_x, \sigma_y]^T, \tag{12.4}$$

followed by

$$\boldsymbol{\mu}_{\text{gen}} \longrightarrow \boldsymbol{\rho}_{\text{synthetic}} \longrightarrow \boldsymbol{\Phi}_{\text{FEM}}. \tag{12.5}$$

The learned operator is trained on $(\boldsymbol{\rho}, \boldsymbol{\Phi})$, not on $(\boldsymbol{\mu}_{\text{gen}}, \boldsymbol{\Phi})$.

13 Hard versus soft enforcement of boundary conditions

13.1 Soft boundary enforcement

The network can be allowed to predict every nodal potential and be penalized with $\mathcal{L}_{\text{D}}$ on tagged electrode nodes. This is general and easy to implement.

13.2 Hard boundary enforcement

For fixed Dirichlet electrodes, hard enforcement is even simpler in the field-output setting: after the network predicts $\boldsymbol{\Phi}_\theta$, overwrite the tagged Dirichlet entries with the exact prescribed values before

evaluating the PDE residual. Equivalently, the output layer can be masked so that only free-node values are learned.

This guarantees exact electrode potentials and reduces the optimization problem to the physically free degrees of freedom. For the first fixed-geometry Module 2 field surrogate, hard Dirichlet enforcement is therefore recommended unless implementation constraints make the soft version simpler.

14 Loss weighting and why it is difficult

14.1 The scale problem

The total field-to-field PINN objective combines several different tasks. For the preferred fixed-geometry baseline, a representative form is:

$$\mathcal{L}_{\text{total}} = \lambda_s \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{PDE}} + \lambda_D \mathcal{L}_{\text{D}} + \lambda_E \mathcal{L}_E. \tag{14.1}$$

A separate Neumann loss can be added when nonzero boundary fluxes are modeled explicitly; homogeneous natural Neumann conditions are already embedded in the discrete FEM weak form. For a time-dependent PINN, an initial-condition term could be added in exactly the same way. Module 2 electrostatics is static, however, so there is no initial-condition loss in the present model.

The numerical values of these losses do not automatically have comparable meanings. For example, an unscaled training run could produce

$$\mathcal{L}_{\text{data}} \sim 10^{-2}, \qquad \mathcal{L}_{\text{PDE}} \sim 10^{5}, \qquad \mathcal{L}_{\text{D}} \sim 10^{-1}. \tag{14.2}$$

If all loss weights are simply set to one, the PDE term can dominate the optimizer because of its numerical scale, not because the PDE is physically more important. Conversely, an oversized boundary term can force an excellent boundary fit while allowing poor interior physics.

This leads to an important distinction:

- **loss normalization** asks how to place fundamentally different errors on comparable numerical scales;
- **loss weighting** asks how strongly the optimizer should prioritize the already-scaled objectives relative to one another.

These two operations should not be conflated. The coefficients λ_i should not be used merely to repair avoidable unit or scale differences.

14.2 Step 1: nondimensionalize and normalize the individual losses

The first defense against loss imbalance is the nondimensionalization introduced earlier in this note. Potential, charge density, and the discrete FEM residual should be expressed in dimensionless or consistently normalized form whenever possible. In particular, Eq. (8.2) divides the discrete residual by a characteristic residual scale r_0 so that the PDE objective does not dominate merely because the FEM equations carry different physical units from the data loss.

After the physical variables are nondimensionalized, each remaining loss may be written in normalized form as

$$\widetilde{\mathcal{L}}_i = \frac{\mathcal{L}_i}{\mathcal{L}_{i,\mathrm{ref}} + \epsilon_L}, \tag{14.3}$$

where $\mathcal{L}_{i,\mathrm{ref}}$ is a characteristic reference scale and ϵ_L is a small positive number that prevents division by zero. A physically derived scale is preferred. If a clean physical reference is unavailable, the initial value of the loss or a representative value measured over the first few batches can be used as an engineering normalization.

For example, if the potential has already been scaled by ϕ_0, then the supervised and Dirichlet losses can be formed directly from dimensionless potential errors. Likewise, the discrete residual can be normalized as

$$\hat{\boldsymbol{r}}_{\mathrm{PDE}} = \frac{\boldsymbol{r}_{\mathrm{PDE}}}{r_0}, \tag{14.4}$$

with r_0 derived from a characteristic source-vector or residual scale. The practical goal is

$$\widetilde{\mathcal{L}_{\mathrm{data}}}, \quad \widetilde{\mathcal{L}_{\mathrm{PDE}}}, \quad \widetilde{\mathcal{L}_{\mathrm{D}}} \quad \text{(and any optional additional terms)} \tag{14.5}$$

to begin training at broadly comparable orders of magnitude rather than being separated by many powers of ten.

Normalization does not decide physical importance. Normalization only removes artificial numerical advantages caused by units or characteristic magnitude. Once the losses are placed on comparable scales, the λ_i coefficients can be used to control the relative training priorities.

14.3 Step 2: begin with equal effective weights

After normalization, the cleanest baseline is

$$\mathcal{L}_{\mathrm{total}} = \widetilde{\mathcal{L}_{\mathrm{data}}} + \widetilde{\mathcal{L}_{\mathrm{PDE}}} + \widetilde{\mathcal{L}_{\mathrm{D}}}, \tag{14.6}$$

which corresponds to

$$\lambda_s = \lambda_r = \lambda_D = 1 \tag{14.7}$$

This is more defensible than assigning an arbitrary factor such as 10 or 100 to one term before observing how the optimizer actually responds. A non-unit fixed weight can still be justified later if validation shows that one constraint deserves additional emphasis, but equal normalized weights provide a reproducible starting point.

14.4 Step 3: measure the gradient contribution of each loss

The optimizer does not update the network directly from the scalar loss values. It updates the neural-network parameters from gradients with respect to the trainable parameter vector $\boldsymbol{\theta}$. Therefore, the relevant quantity for objective i is

$$\boldsymbol{g}_i = \nabla_{\boldsymbol{\theta}} \widetilde{\mathcal{L}}_i, \tag{14.8}$$

with gradient norm

$$g_i = \left\| \nabla_{\boldsymbol{\theta}} \widetilde{\mathcal{L}}_i \right\|_2 . \tag{14.9}$$

The weighted contribution of this term to the optimizer is approximately characterized by

$$G_i = \lambda_i g_i. \tag{14.10}$$

This is important because two normalized losses can have similar scalar values while producing very different parameter gradients. For example,

$$g_{\text{data}} = 10^{-2}, \qquad g_{\text{PDE}} = 10 \tag{14.11}$$

means that, with equal weights, the PDE term can exert roughly three orders of magnitude more influence on a gradient-based update. Monitoring only the loss values would not reveal this imbalance.

A useful engineering target is therefore not merely

$$\widetilde{\mathcal{L}}_i \sim \widetilde{\mathcal{L}}_j, \tag{14.12}$$

but also to avoid persistent extreme imbalance in the weighted gradient contributions,

$$\lambda_i g_i. \tag{14.13}$$

14.5 Step 4: adaptive gradient-balanced loss weighting

Instead of selecting fixed λ_i values by trial and error, Module 2 can use adaptive loss weighting. The basic idea is simple: if one objective produces an excessively large gradient, reduce its weight; if another produces a very small gradient and is being neglected, increase its weight.

Let there be M active loss terms. At training iteration k, compute or periodically estimate

$$g_i^{(k)} = \left\| \nabla_{\boldsymbol{\theta}} \widetilde{\mathcal{L}}_i^{(k)} \right\|_2 . \tag{14.14}$$

Because minibatch gradients can be noisy, first form a smoothed gradient norm,

$$\bar{g}_i^{(k)} = \beta \bar{g}_i^{(k-1)} + (1-\beta) g_i^{(k)}, \qquad 0 \le \beta < 1. \tag{14.15}$$

An inverse-gradient target weight can then be defined as

$$\lambda_{i,*}^{(k)} = \frac{\left(\bar{g}_i^{(k)} + \epsilon_g\right)^{-1}}{\frac{1}{M}\sum_{j=1}^{M} \left(\bar{g}_j^{(k)} + \epsilon_g\right)^{-1}}, \tag{14.16}$$

where ϵ_g prevents division by zero. The denominator normalizes the weights so that their mean is approximately one. The corresponding weighted gradient magnitudes tend toward a comparable scale because

$$\lambda_{i,*}^{(k)} \bar{g}_i^{(k)} \tag{14.17}$$

is approximately equalized across the active objectives.

To avoid abrupt oscillations, the actual weight can be updated gradually:

$$\lambda_i^{(k+1)} = (1-\alpha)\lambda_i^{(k)} + \alpha \lambda_{i,*}^{(k)}, \qquad 0 < \alpha \ll 1. \tag{14.18}$$

In practice, the λ_i should also be clipped to a reasonable interval,

$$\lambda_{\min} \le \lambda_i \le \lambda_{\max}, \tag{14.19}$$

so that a temporarily tiny gradient does not produce an enormous weight. The adaptive calculation does not need to be performed at every optimizer step; updating every several iterations or every epoch can reduce overhead and noise.

This procedure is not the only possible adaptive weighting algorithm. Its value is that it expresses the central principle transparently: *balance the influence of the different objectives on the parameter update, not merely their raw scalar loss values.*

14.6 Why adaptive weighting is useful for Module 2

The relative difficulty of the Module 2 objectives changes during training. Early in training, the data mismatch may be large. Later, the network may fit FEM values well while the Poisson residual remains comparatively difficult. In another stage, the interior physics may be accurate while a boundary error persists. Therefore, the most useful values of λ_i need not remain constant over the

entire optimization.

Adaptive weighting allows

$$\lambda_i = \lambda_i(k) \tag{14.20}$$

to evolve with the training state. If the PDE residual is being neglected, its effective weight can rise. If the PDE gradient later dominates all other updates, its weight can fall. The purpose is not to force the scalar losses to be identical; the purpose is to prevent one objective from monopolizing the gradient-based optimizer solely because of numerical conditioning.

14.7 Diagnostics and safeguards

During training, record more than the total loss. At minimum, plot

$$\widetilde{\mathcal{L}_{\text{data}}}, \qquad \widetilde{\mathcal{L}_{\text{PDE}}}, \qquad \widetilde{\mathcal{L}_{\text{D}}}, \qquad \widetilde{\mathcal{L}_{E}}, \tag{14.21}$$

together with the adaptive weights

$$\lambda_s, \qquad \lambda_r, \qquad \lambda_D, \qquad \lambda_E \tag{14.22}$$

and, when practical, the weighted gradient norms

$$G_s, \qquad G_r, \qquad G_D, \qquad G_E. \tag{14.23}$$

A decreasing total loss can otherwise hide a boundary loss that has stalled or a PDE residual that remains physically unacceptable.

Adaptive weighting also does not remove the need for validation. A mathematically balanced optimizer can still converge to a poor surrogate. The final weighting strategy should therefore be judged using held-out FEM cases, analytical verification cases, boundary errors, and Poisson-residual maps.

A small fixed-weight parameter sweep can also be useful as a sensitivity study. For example, after establishing a normalized baseline, selected λ_i values can be perturbed over a modest range and the resulting validation error and residual error compared. If the final surrogate changes dramatically under small weight changes, the training procedure is not yet robust.

14.8 Recommended Module 2 procedure

The recommended workflow for the first physics-informed surrogate is

$$\boxed{\begin{array}{c}\text{nondimensionalize} \longrightarrow \text{normalize losses} \longrightarrow \text{start with equal weights} \\ \text{measure gradient balance} \longrightarrow \text{adapt weights if needed}\end{array}} \tag{14.24}$$

with validation performed throughout training.

For the first implementation, it is reasonable to begin with the equal-weight normalized objective in Eq. (14.6). The gradient norms should then be recorded. If one term persistently dominates or is persistently ignored, activate the adaptive update in Eqs. (14.15)–(14.18). This gives Module 2 a consistent engineering procedure for selecting loss weights rather than relying on subjective trial-and-error coefficients.

Key takeaway. First make the losses numerically comparable; only then decide how strongly each objective should influence training. For a gradient-based PINN optimizer, adaptive weighting should be based on the gradient contribution of each normalized loss, because those gradients are what actually update the neural-network weights and biases.

15 Validation metrics

The neural surrogate should be validated against FEM and analytical cases using complete held-out fields. Useful metrics include the relative potential-field error

$$\epsilon_\phi = \frac{\|\mathbf{\Phi}_\theta - \mathbf{\Phi}_{\mathrm{FEM}}\|_2}{\|\mathbf{\Phi}_{\mathrm{FEM}}\|_2 + \epsilon}, \tag{15.1}$$

the relative electric-field error

$$\epsilon_E = \frac{\|\mathbf{E}_\theta - \mathbf{E}_{\mathrm{FEM}}\|_2}{\|\mathbf{E}_{\mathrm{FEM}}\|_2 + \epsilon}, \tag{15.2}$$

and the normalized free-node FEM residual

$$\epsilon_R = \frac{\|\boldsymbol{r}_{\mathrm{PDE}}\|_2}{r_0\sqrt{N_F} + \epsilon}. \tag{15.3}$$

Here ϵ is a small number used to avoid division by zero.

Field plots should include:

- input total charge-density field $\rho(x, y)$;
- FEM potential $\phi_{\mathrm{FEM}}(x, y)$;
- field-surrogate potential $\phi_\theta(x, y)$;
- error field $\phi_\theta - \phi_{\mathrm{FEM}}$;
- FEM and predicted electric-field magnitude;
- nodal or elementwise discrete residual map.

16 How the Module 2 field surrogate connects to the existing MATLAB solver

16.1 Existing Module 2 reference role

The FEM implementation remains the reference generator. Its new canonical single-case interface is an explicit nodal charge field:

$$\boldsymbol{\rho} \longrightarrow \text{FEM} \longrightarrow \boldsymbol{\Phi}_{\text{FEM}}, \mathbf{E}_{\text{FEM}}. \tag{16.1}$$

The same $\boldsymbol{\rho}$ array is saved as the neural-surrogate input. This eliminates an unnecessary compression step between the physics solver and the learning model.

The Gaussian routine now has only this role:

$$\boldsymbol{\mu}_{\text{gen}} \longrightarrow \text{synthetic charge-field generator} \longrightarrow \boldsymbol{\rho}. \tag{16.2}$$

It is one source of training cases, not the Module 2 interface.

16.2 Current code state and migration boundary

The FEM code has been refactored to accept arbitrary nodal $\boldsymbol{\rho}$ fields and to save field-to-field batch datasets. The executable files under `matlab/pinn/` still implement the older pointwise/single-case PINN prototype. They are intentionally retained as regression code until the field-to-field neural architecture is implemented.

Therefore the project currently has a clean migration boundary:

FEM/data contract: field-to-field now	neural-network executable: migrate next

(16.3)

16.3 Target adjacent neural files

The later PINN-code migration should preserve the existing top-level organization but change the internal tensor contract. A possible structure is:

```
matlab/
  main_module2_electrostatics.m
  main_module2_batch_fem_dataset.m
  main_module2_pinn_electrostatics.m      field-to-field driver after migration
  pinn/
    module2_pinn_network.m                field operator architecture
```

`module2_pinn_loss.m`	`data + discrete FEM physics residual`
`module2_pinn_train.m`	`complete-field minibatch training`
`module2_pinn_predict.m`	`rho field -> phi field`
`module2_pinn_make_dataset.m`	`load v2 rho/phi pairs`
`module2_pinn_verify.m`	`held-out field verification`

17 Recommended staged implementation

17.1 Stage 1: validate arbitrary FEM field input

Verify that non-Gaussian nodal charge arrays, uniform arrays, and Gaussian-generated arrays all enter the same FEM solver path and produce finite, converged solutions. This stage is now represented by explicit-field regression tests.

17.2 Stage 2: build a diverse field dataset

Use the Gaussian generator as the first controlled pilot, but store the complete $\boldsymbol{\rho}$ and $\boldsymbol{\Phi}_{\mathrm{FEM}}$ arrays. Then add non-Gaussian and physically imported fields so that the training distribution is not restricted to one analytic family.

17.3 Stage 3: supervised field-to-field baseline

Train $\mathcal{G}_\theta : \boldsymbol{\rho} \mapsto \boldsymbol{\Phi}_\theta$ using only $\mathcal{L}_{\mathrm{data}}$. Evaluate on complete held-out charge fields. This establishes the architecture's raw approximation capability.

17.4 Stage 4: add the discrete physics loss

Evaluate Eq. (4.5) for each predicted field. Combine $\mathcal{L}_{\mathrm{PDE}}$ with the data and boundary objectives, normalize the individual losses, then use the adaptive gradient-balanced weighting procedure developed in Chapter 14.

17.5 Stage 5: expand to coupled physical fields

Replace or supplement Gaussian synthetic fields with total charge maps constructed from Module 3 and Module 5 outputs:

$$\{C_i(x,y), p(x,y), n(x,y), N_D^+(x,y), N_A^-(x,y)\} \longrightarrow \rho(x,y) \longrightarrow \phi_\theta(x,y). \tag{17.1}$$

This is the key route toward Module 6 multiphysics coupling.

18 How this becomes a surrogate speedup

A field-to-field network has a larger input and output than the earlier low-dimensional parameter surrogate, but the same amortization argument applies. Let T_{FEM} be the cost of one reference field solve, N_{train} the number of training field pairs, and T_{train} the network training cost. Then

$$T_{\mathrm{upfront}} = N_{\mathrm{train}} T_{\mathrm{FEM}} + T_{\mathrm{train}}. \tag{18.1}$$

If one trained field inference costs T_{NN}, the surrogate is advantageous after enough future field queries that

$$N_{\mathrm{query}} T_{\mathrm{FEM}} > T_{\mathrm{upfront}} + N_{\mathrm{query}} T_{\mathrm{NN}}. \tag{18.2}$$

The benefit is greatest when Module 2 must be evaluated repeatedly inside parameter sweeps, uncertainty propagation, optimization, or iterative multiphysics coupling.

19 Physical interpretation of the trained surrogate

The trained model approximates an electrostatic *solution operator*, not a new physical law:

$$\mathcal{G} : \rho(\boldsymbol{x}) \mapsto \phi(\boldsymbol{x}). \tag{19.1}$$

The important outputs and diagnostics are

$$\boldsymbol{\Phi}_\theta, \qquad \mathbf{E}_\theta, \qquad \boldsymbol{r}_{\mathrm{PDE}} = K_{FF}\boldsymbol{\Phi}_{\theta,F} + K_{FD}\boldsymbol{\Phi}_D - \boldsymbol{b}_F(\boldsymbol{\rho}). \tag{19.2}$$

A good surrogate must perform well on complete unseen charge fields and must remain close to the discrete Poisson manifold, not merely reproduce training examples.

20 Common failure modes

20.1 Boundary-condition failure

If Dirichlet values are learned softly, the network may fit interior data while drifting at the electrodes. Hard masking removes this failure mode for fixed boundary conditions.

20.2 Physics-residual failure

A network can have small supervised error while producing a significant FEM residual. This means the predicted field is close numerically to reference data but is not sufficiently consistent with the discrete electrostatic operator.

20.3 Loss imbalance

Data, PDE, and boundary objectives can drive very different parameter-gradient magnitudes. Nondimensionalize first and then use the adaptive gradient-balancing procedure in Chapter 14.

20.4 Poor field-distribution coverage

Training only on smooth single Gaussians can produce excellent interpolation within that family while failing on multimodal or irregular maps. The training set must eventually contain the spatial structures expected from Modules 3 and 5.

20.5 Grid or mesh representation mismatch

A CNN requires structured arrays, while the current Module 9 FEM uses a triangular mesh. If fields are rasterized, interpolation and geometry masks must be validated carefully. A mesh/graph operator avoids this conversion but requires a different network implementation.

20.6 Sharp localized charge distributions

Narrow charge structures create localized potential curvature. The spatial representation, mesh resolution, and network receptive field must all be sufficient to resolve those structures.

21 How to present Module 2_PINN in an interview or report

The clean presentation narrative is:

1. Module 2 receives a complete total charge-density field and solves Poisson electrostatics for the complete potential field.
2. Gaussian parameters are retained only as one synthetic training-field generator; they are not the surrogate input contract.
3. The neural model learns the operator $\boldsymbol{\rho} \mapsto \boldsymbol{\Phi}$ over complete spatial configurations.
4. The reference FEM weak form supplies a differentiable discrete physics residual $K\boldsymbol{\Phi}_\theta - \boldsymbol{b}(\boldsymbol{\rho})$.
5. Supervised field error, physics residual, and boundary constraints are normalized and adaptively balanced during training.
6. Validation is performed on complete unseen charge fields, including non-Gaussian cases.

The most important figure set is:

- input charge-density field ρ;
- FEM potential versus neural potential;
- potential error field;
- electric-field comparison;
- discrete physics-residual field or nodal residual map;
- held-out error and inference-time comparison.

22 Relationship to future Module 6 PINN coupling

The field contract is especially useful for multiphysics coupling because upstream modules can exchange spatial states without forcing them into low-dimensional Gaussian summaries. A future iteration may follow

$$\text{Module 3 defect fields} \; + \; \text{Module 5 carrier fields} \longrightarrow \boldsymbol{\rho} \longrightarrow \mathcal{G}_{2,\theta}(\boldsymbol{\rho}) \longrightarrow \boldsymbol{\Phi}, \mathbf{E}, \tag{22.1}$$

with those electrostatic fields then fed back into transport or other coupled modules.

The exact coupled architecture can be developed later. The important Module 2 decision has already been made: its learned interface should preserve the spatial field information required by multiphysics coupling.

23 Summary of the Module 2_PINN model

The revised core Module 2 surrogate is

$$\boxed{\mathcal{G}_\theta : \boldsymbol{\rho} \mapsto \boldsymbol{\Phi}_\theta} \tag{23.1}$$

with total charge assembled from spatial component fields according to

$$\boldsymbol{\rho} = q\left(\boldsymbol{p} - \boldsymbol{n} + \boldsymbol{N}_D^+ - \boldsymbol{N}_A^- + \sum_i \boldsymbol{z}_i \odot \boldsymbol{C}_i\right). \tag{23.2}$$

For the fixed FEM mesh, the preferred physics residual is

$$\boxed{\boldsymbol{r}_{\mathrm{PDE}} = K_{FF}\boldsymbol{\Phi}_{\theta,F} + K_{FD}\boldsymbol{\Phi}_D - \boldsymbol{b}_F(\boldsymbol{\rho})} \tag{23.3}$$

and the training objective is

$$\boxed{\mathcal{L}_{\text{total}} = \lambda_{\text{data}}\widetilde{\mathcal{L}_{\text{data}}} + \lambda_{\text{PDE}}\widetilde{\mathcal{L}_{\text{PDE}}} + \lambda_D\widetilde{\mathcal{L}_{\text{D}}} + \lambda_E\widetilde{\mathcal{L}_E}} \tag{23.4}$$

with loss normalization followed, when needed, by adaptive gradient-balanced weighting.

The old Gaussian parameters are now only a method for generating controlled $\boldsymbol{\rho}$ fields. This makes the Module 2 interface compatible with arbitrary spatial charge maps and provides a direct path toward later Module 3/5/6 coupling.

24 References for further study

The following references are useful background for the concepts used in this note.

1. M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 2019.

2. G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, 2021.

3. J. N. Reddy, *An Introduction to the Finite Element Method.* Useful for the weak-form and FEM background that underlies the reference Module 2 solver.

4. S. M. Sze and K. K. Ng, *Physics of Semiconductor Devices.* Useful for semiconductor electrostatics, dopant charge, and carrier transport background.

5. The existing `module2_2d_electrostatics_defect_space_charge.tex` note in this project. That document remains the reference physics derivation for Module 2 electrostatics and FEM.